出力・可視化の方針

最終更新: 2026-08-23（HBN240018）

最終的に作りたい出力の構成と、そのために**各計算段階で何を算出・蓄積しておく必要があるか**を まとめた文書。**これから書くコードは常にここに必要な情報を落とせる形にしておくこと。**

★当初の 6 種はすべて実装済み（2026-08-21 時点）。そのあと依頼で **音圧レベル・STI・測定点の配置図** が加わり、CSV の上に**Excel 1 枚**をかぶせる形に なった。下の表がいまの全体像。

想定している出力構成

#	出力	用途	データ源	状況
①	**音線の可視化**	反射経路を線で確認する	音線法（③）の軌跡	**完了**（`view_rays.py --mode rays`）
②	**音粒子の可視化（動画）**	離散化時間ごとに粒子が飛ぶ様子。音の広がりを見る	音線法（③）の軌跡	**完了**（`--mode particles`、GIF 出力あり）
③	**インパルス応答**	—	⑥ impulse.py	**完了**（CSV + `impulse_response.png`）
④	**各種音響指標**	T30, EDT, C50, D50 など	③のIRから	**完了**（`reverberation.png` / `clarity.png`）
⑤	**伝搬方向確認（図）**	受音点に人の頭の絵を置き、どの方向成分が多いか	⑤ バックトレースの出力	**完了**（`direction.png`）
⑥	**モード分布**	受音点における	③のIRから／別途	**完了**（`modes.png`）
⑥-2	**モードの積み上げと吸音の効果**	経路差から強め合う周波数を数え、吸音の有無で比べる	⑤ バックトレースの出力	**完了**（`mode_buildup.png`）
⑦	**音圧レベル（帯域別）**	絶対値の L_p。PWL 未入力なら相対値	⑤ バックトレースの出力	**完了**（`spl.csv` / `spl.png`。依頼 08-21）
⑧	**STI（音声伝送指数）**	明瞭度。IEC 60268-16	⑤ バックトレースの出力	**完了**（`sti.csv` / `sti.png`。依頼 08-21）
⑨	**測定点の配置**	どれがどの点で、どこを向いているか	入力（音源・受音点・正面方位）	**完了**（`測定点.csv` / `points.png`。依頼 08-23）
⑩	**虚音源の可視化**	虚音源と受音点を結ぶ線。反射のからくりを目で見る	⑤ のパルス列（**計算はやり直さない**）	**完了**（音線・音粒子と同じ窓の **Tab**。依頼 08-24）

（番号の①〜⑨は出力の識別番号。パイプラインの①〜⑦とは別物）

表として残すもの（機械が読む CSV）

ファイル	中身	置き場
`pulses.csv` / `ir.csv`	パルス列・インパルス応答	`結果/recN/`
`rt.csv` / `decay.csv` / `clarity.csv`	残響指標・減衰曲線・明瞭度	`結果/recN/`
`spl.csv` / `sti.csv`	音圧レベル・STI	`結果/recN/`
`吸音率と理論値.csv`	材料別の吸音率 → 平均吸音率 → 残響時間理論値	`結果/`（**受音点に依らない**）
`測定点.csv`	音源・受音点の座標と正面方位・音源からの距離	`結果/`（**条件にも依らない**）
`まとめ_*.csv`（4 枚）	全測定点を 1 枚に（残響時間 / 明瞭度 / 音圧レベル / STI）	`結果/`
`まとめ_条件比較.csv`	条件を横に並べた比較（一括計算したとき）	`結果/`
`経路.npz`	経路の幾何（反射面の並びと入射角）。**使い回す**ので消さない	`結果/recN/`
`raylog.npz`	音線の軌跡（①②の可視化用）	`結果/`（受音点に依らない）

人が読むもの（Excel 1 枚）

`結果/<室>_結果一式.xlsx`（`workbook.py`）。**CSV を読み直すだけで計算はしない。** シートは 概要／残響時間／明瞭度／音圧レベル／STI／吸音率と理論値／**測定点**／材料条件表 （＋一括計算したときは 条件比較）。グラフ付き。`--template 雛形.xlsx` で 雛形の書式・グラフ・ロゴを残したまま値だけ流し込める。

★CSV（機械が読む）と Excel（人が読む）は役割を分ける（2026-08-21 ユーザー判断）。 CSV は全部残したまま、その上に Excel をかぶせている。

モデルビューア（先に作成済み・2 種類）

上記 6 種の前に、**読み込んだモデルを確認するビューア**を作ってある。表示内容は同じで、 実装方式が違う 2 本を**並存させている**。

```
cd geosim
python view_model.py     ..\test2.dxf --absorption ..\absorption.csv   # HTML + WebGL
python view_model_gui.py ..\test2.dxf --absorption ..\absorption.csv   # PyVista ウィンドウ
```

- 三角形要素（辺を描くので分割が見える）／法線ベクトル（矢印）／レイヤ別の色分け
- **法線の裏側を赤で塗る**ので、向きの誤りが一目で分かる

- 回転／拡大縮小／平行移動／視点プリセット
- 音源・受音点、読み込み結果のサマリ

	`view_model.py`	`view_model_gui.py`
方式	自己完結 HTML + WebGL を書き出してブラウザで開く	PyVista（VTK）のネイティブウィンドウ
依存	なし（標準ライブラリ + numpy）	pyvista + vtk
強み	**相手に環境構築を求めずに共有できる**	**Python から直接操作できる**
発展先	Web ベース GUI	①②の重ね描き、デスクトップ GUI

HTML 版を先に作ったのは、当時この環境に matplotlib / pyvista が入っておらず （`tkinter` のみ)、`pip install` を前提にしたくなかったため。 2026-08-14 に Python 3.10.11 の venv を切って pyvista を導入し、ネイティブ版を追加した。 **HTML 版は捨てていない**。共有用途では今も HTML 版のほうが便利なため。

したがって GUI（G-7）は **Web ベース / デスクトップのどちらに倒しても土台がある**状態。

音線・音粒子ビューア（①② 実装済み）

`geosim/view_rays.py`。上のモデルビューア（PyVista 版）に重ねて表示する。

```
cd geosim
# ① 音線（受音した経路の初期反射だけ、到来時刻で色分け）
python view_rays.py ..\test.dxf ..\結果\test_raylog.npz --received-only --max-reflection 3 --color
time
# ② 音粒子（スペース 再生/停止、← → コマ送り、スライダで時刻指定）
python view_rays.py ..\test.dxf ..\結果\test_raylog.npz --mode particles
# ② を GIF に
python view_rays.py ..\test.dxf ..\結果\test_raylog.npz --mode particles --movie 広がり.gif
```

やってみて分かったこと

- **音線は本数を絞らないと読めない。** 小さい室で 250 本 × 13 反射を描くと

  線が重なって何も分からなかった。既定を 80 本にし、
  `--max-reflection` で**折れ線を途中で打ち切れる**ようにした。
  「その回数以下の音線だけ残す」ではないのが要点で、閉じた室では
  全音線が上限まで反射するため絞り込みでは何も残らない
- **受音した経路だけを見るのがいちばん分かりやすい。** `--received-only`
- **音粒子は全音線ぶん出したほうがよい。** こちらは点なので重なっても密度として読める。

  むしろ本数が多いほど「広がり」が見える
- 動画は追加の依存を増やさず **Pillow で GIF** にした

（60 コマ 960×720 で 3.3 MB。長くするなら間引きかフレーム数の調整が要る）

- **壁の透過はレイヤごとに変えられるようにした**（ユーザー要望、2026-08-14）。

  左の縦スライダ、`Tab` で対象切替、`m` でモデル表示 ON/OFF。
  起動時指定は `--opacity` と `--layer-opacity "1=0.6,2=0.05"`
- **再生は VTK のタイマーでは動かなかった**。`add_timer_event` が発火せず、

  「コマ送りは効くのに再生されない」状態だった。
  `show(interactive_update=True)` + 自前ループに変更して解決

GUI の全体設計（G-7）は後回しにしている（ユーザー判断）。 ここは「見るための最小限」に留めてあり、将来 GUI に組み込むときは `RayLog` と `ParticleAnimation` をそのまま使える形にしてある。

重要な前提：最終的な計算（到来時刻・エネルギー）は音線法ではなく虚音源バックトレースで行うが、**音線法の段階のデータも捨てずに蓄積する**。出力①②は音線法ベースのデータを使う。

出力①② のためのデータ — 音線軌跡

対応コード: `geosim/ray_recorder.py`、`loop_reflectionmesh.loop(..., recorder=...)`

なぜ本線と分けたか

音線追跡ループの「本線」の出力は**受音した経路の反射面 ID 列**（元コード `traceff`）だけ。 一方、可視化で見たいのは「音がどう広がるか」なので、**受音しなかった音線も必要**。 目的が違うので同じ配列に混ぜず、`RayRecorder` を副チャンネルとして渡す設計にした。 `recorder=None` なら一切記録しないので、本番計算の速度・メモリには影響しない。

記録している内容（`RayTrajectory`）

項目	形状	内容	使い道
`ray_index`	int	間引き前の元の音線番号	再現性・デバッグ
`direction`	(3,)	音源から出たときの初期方向	出射方向で色分けする等
`nodes`	(m+1, 3)	節点座標（先頭が音源、以降が各反射点）	**①折れ線として描画**
`distances`	(m+1,)	音源からの累積距離 [m]	**②時刻の算出**
`mesh_ids`	(m,)	各セグメント終端で反射した面 ID	壁ごとの色分け
`energies`	(m+1, 6)	各節点でのバンド別エネルギー	**線・粒子の減衰表現**
`receive_steps`	(r,)	受音したセグメント番号 k	受音経路のハイライト
`termination`	str	`'no_hit'` / `'nref'`	打ち切り理由の把握

- 時刻は `traj.times(c) = distances / c` で得る。
- 音粒子アニメーションは `traj.position_at(t, c)` を全軌跡について毎フレーム呼べばよい

  （セグメント間を線形補間して位置を返す。未出発／消滅後は `None`）。
- 保存は `recorder.save_npz(path)`。可変長なので**連結＋オフセット方式**で持つ

  （`nodes[node_offsets[i]:node_offsets[i+1]]` で i 本目が取れる）。

間引きの方針

100 万本飛ばしても全部は描かないので間引きが要る。**「総数比 1/100」ではなく「絶対本数の上限＋等間隔ストライド」を既定**にした（`max_rays`、既定 2000 本）。

理由:

- **総数比だと描画本数が安定しない**。1/100 は 100 万本なら 1 万本（重くて描画が破綻する）、

  1000 本なら 10 本（少なすぎて広がりが見えない）。可視化の品質は「何本描くか」で決まるので、そこを直接指定するほうが素直。
- **絶対本数なら描画性能が音線数に依存しない**。`stride = total // max_rays` で自動追従する。
- 音線数を変えて計算しても、**見た目の密度が変わらないので比較しやすい**。

対話的に「もっと細かく／粗く」を切り替えたい場面は出てくるので、`max_rays` は GUI 側から渡せるパラメータにしておく想定。

> **注意**：Fibonacci 螺旋は添字 `i` に対して高さ `z` が単調増加、方位角が黄金角ずつ回る配置なので、
> 等間隔ストライドで抜いても `z` は一様なまま（＝等立体角は保たれる）。ただし
> ストライド幅と黄金角の関係次第では方位角に縞模様が出る可能性がある。
> 描画してみて偏って見えたら、固定シードのランダム抽出に切り替える。

出力⑤ のためのデータ — 伝搬方向

対応コード: `loop_noredundancy.py`（バックトレース）

元コード 1080 行の出力にすでに含まれている。

```
ktmp, rtime, -vtgt(1:3), enertmp(1:6)
 ↑     ↑        ↑            ↑
反射回数 到来時刻   到来方向      バンド別エネルギー
```

到来方向ベクトル（3〜5 列目）がそのまま出力⑤ の素材になる。 受音点を原点として、この方向ごとにエネルギーを積み上げれば方向分布が描ける。 人の頭の絵を置くなら、頭部座標系（正面・上方向）を別途

入力として持たせる必要がある。

- 時間で区切った方向分布（初期反射だけ／後部残響だけ）も見たくなる見込みなので、

  到来時刻とセットで保持しておくこと。
- バンド別に分けられるようにしておくこと（低音と高音で方向性が違う）。

★図の向きの約束（2026-08-23 に取り違えが見つかった）

到来方向は `PulseList.direction`（**受音点から虚音源を見込む単位ベクトル**）。 図は**頭を基準にした相対方位**で描き、世界座標の X・Y ではない。

相対方位	図の位置	計算
正面 0°	**上**	$(x, y) = (-\sin a,\ \cos a)$
左 90°	図の**左**	真上から見て正面が上なら、その人の左手は図の左を指す
後ろ 180°	下	
右 270°	図の**右**	

★人の絵（鼻）も上に向ける。分布・ラベル・直接音の矢印は上を正面として描いて いたのに、人の絵の鼻だけ +X（図の右）を向いていて、**絵だけ 90° ずれていた。** データと矢印は正しかったので、人の絵を基準に読むと直接音が違う方向から来ているように 見える、という分かりにくい誤りだった（`plots._head_patch`。テスト [18] で固定）。

- 上下は見ない（水平面へ投影する）。実務では水平面で足りる、というユーザー判断
- 頭の正面方位は**受音点ごとに持てる**（`project.head_azimuth`）。図の下に値を書く

出力⑩ のためのデータ — 虚音源（依頼 2026-08-24）

> 虚音源を可視化して見れるようにしたい。
> ・虚音源から指定した受音点（画面上で指定）を結ぶ線を描く。
> ・音線と同じように、N 回反射まで表示。N 回目反射のみを表示というのも欲しいので、
> 反射回数の開始と終了を設定できるようにすれば良いかと思います。

★新しいデータを蓄積する必要はなかった。パルス列がすでに持っている。

虚音源の位置 = 受音点 + 到来方向 × 距離

到来方向を**単位ベクトルにして距離を別に持つ**という決め（`loop_noredundancy` 冒頭。元コードは `-vtgt` の 1 つに混ぜていた）が、ここで効いた。 `結果/recN/<室>_pulses.csv` を読むだけなので**計算をやり直さない。**

- **反射回数は「開始」と「終了」で絞る。**同じ値にすれば「N 回目のみ」

（ユーザー要望どおり上限 1 つではなく範囲にした）

- **受音点は画面のスライダと `w` キーで選ぶ。**選んでいる点だけ濃く描く

  （モデル上のクリックは難しいというユーザー指摘があるので、
  操作はスライダとキーで完結させる方針。`ray_filter` と同じ）
- 本数で絞るときは**エネルギーの大きい順**（適当に間引くと効いている虚音源が落ちる）
- **★カメラを虚音源に合わせると室が点になる**（高次は室の外へ 3 km 離れる）。

  `h`（室に寄る）と `r`（全部が入る）を用意した
- 「N 回目のみ」だと色の値が全部同じになるので、そのときは**1 色で塗る**

  （カラーバーが「1.0 1.0 1.0」と並んで意味が無い）

出力⑨ のためのデータ — 測定点の配置（依頼 2026-08-23）

> どれがどの点で、どの方向を向いているかがわからない。表形式で音源位置と、
> 受音点名と座標情報、向いている方向を記載して。平面図など図を見てわかる資料も。
> 平面上被っている場合もあるので、平面と2方向の立面で。

素材は**入力そのもの**（音源・受音点・正面方位）なので計算は要らない。要点は約束の方。

- **名前は `結果/recN/` に合わせる**（`src1` / `rec1`…）。**表とフォルダが対応する**のが

  この出力の目的。何点目かは `Project.receiver_index` が持つ
- **受音点にも条件（吸音材）にも依らない**ので `結果/` `図/` 直下・条件名なし
- 図は**平面（X-Y）＋立面 2 方向（X-Z / Y-Z）**の 3 面。**★平面で重なる点があるため**
- **★同じ位置に重なる点はラベルを縦にずらす**（重なると名前が読めず、

  立面で見分けても照合できない）
- **正面方向の矢印は平面図だけ。**方位は水平面の量なので、立面に描くと

  見た目の長さが向きによって変わって誤解を生む
- 室の形は**同一平面パッチの外周だけ**を線で描く（三角形の辺を全部引くと網目になる）
- 音源からの距離を表に入れる（**逆二乗の確認にそのまま使える**）

対応コード: `project.write_points_csv()` / `plots.measurement_points()`

出力③④⑥ のためのデータ

- **③ インパルス応答** … `impulse.py` の出力そのもの。CSV で時間ベクトル付き
- **④ 各種音響指標** … ③の IR から算出。Schroeder 積分 → T30 / EDT、

  初期/後期のエネルギー比 → C50 / D50。**バンド別に出せるようにしておく**
- **⑥ モード分布** … IR の FFT。低域の固有周波数分布を見る用途。

  書籍 2.1「波動音響理論」（固有振動・直方体音場・固有周波数分布）が理論的な下敷き
- **⑥-2 モードの積み上げと吸音の効果** … ⑤ のパルス列を**位相込みで足す**。

  到来時刻の差＝経路差なので、経路差が波長の整数倍になる周波数で強め合う。
  減衰を考えなければ `|H(f)|` は「その周波数で同位相に重なった経路の本数」そのもの。
  重みを `1` → `1/d`（完全反射）→ `√E/d`（設計の吸音）と変えて 3 本描き、
  後ろ 2 本の差を吸音の効果として塗る。
  パルスのエネルギーには面の吸音しか入っていない（距離減衰と空気吸収は
  `impulse.py` 側で掛ける約束）ので、3 本をきれいに分離できる。
  計算は「時刻をヒストグラムにして FFT」なので O(n log n)。処理は増えない

これから実装するときのチェックリスト

新しい計算段階を書くときは、以下を満たしているか確認する。

- [ ] **時刻を保持しているか。**②の動画も⑤の時間区切りも時刻がないと作れない
- [ ] **バンド別のまま保持しているか。**合算するのは最後の描画段階でよい
- [ ] **受音しなかったデータも要るか考えたか。**①②は要る、③④⑤⑥は不要
- [ ] **間引き・保存が本線の速度に影響しない形か**（optional な副チャンネルにする）
- [ ] **GUI からパラメータを渡せる形か**（ハードコードしない）
- [ ] **受音点に依らない量を受音点ごとに回していないか**（`結果/` 直下に置き、

  1 点目の結果を配る。統計残響式・音線追跡・経路の幾何がこれ）
- [ ] **表は「周波数を横」に並べたか**（`geosim/table.py` の共通ルール）
- [ ] **ファイル名の頭に対象室＋条件名を付けたか**（`result_path` / `figure_path` を通す）。

  ★条件に依らないものには条件名を付けない（`ROOM_SCOPED_RESULTS`）
- [ ] **Excel に載せるとき数値を数値として書いたか**（`SHEET_LAYOUT` の `text` 列数）

決まったこと（かつての「未決事項」）

かつての論点	どう決まったか
GUI のフレームワーク	**PyVista のネイティブウィンドウ**（3D）＋ **tkinter**（入力フォーム・進捗）。`pyvistaqt` / `PySide6` は**入れていない**。左パネルは 3D の中に自前で組んでいる（`view_model_gui.ControlPanel`）
音粒子アニメーションの作り方	**Python 側で書き出す**（`view_rays.py`）。★2026-08-24 に **GIF → MP4（H.264）**へ（「再生・停止が押せる形式、できれば iPad でも見れる形式」）。最初は停止・離散化時間 1 ms・再生速度と動画の長さはスピンボックスで決める
条件違いの結果を見比べる画角	**画角をファイルに残して読み直す**（`view_camera.py`。`プロジェクトフォルダ/視点.json`）。★2026-08-24 ユーザー要望「条件違いで計算した結果を比較したい時に、音線の結果など画角などを合わせたい」。条件を変えると画面は開き直しになるので、視点が毎回変わって並べられなかった。カメラだけでなく**左パネルの数値と開いていたタブ**も残す（この画面に無い設定は飛ばす）
逆二乗（無響室・半無響室の確認）	**別ルートで出す**（`inverse_square.py`）。2026-08-21 に「音圧レベルの表に自由音場の値と差は入れない」と決めた判断はそのまま。★2026-08-24 ユーザー要望「逆二乗用にテンプレートを常備しておく必要はないですが、これは別途別ルートとして逆二乗だしてほしい」。受音点のレイヤ（`rec1` `rec2` `rec3`）を測線として、理想の $1/r^2$ からのずれをCSV と図にする。計算はやり直さない
面の上に置いた音源	**半球に放射し、載っている面の 1 次反射は起こさない**（`source_placement.py`）。指向係数 $Q = 4\pi / \Omega = 2$（+3 dB）を掛ける＝床置き音源のパワー W が 2π に出る形。剛な取り付け面を仮定（ISO 3744/3745 と同じ扱い）
出力フォーマットの統一	**統一しない。役割で分ける**（上記。CSV ＝ 機械が読む、Excel ＝ 人が読む、npz ＝ 中間データ）
頭部座標系の入力方法	**3D 画面のスライダ**で決める（`face_editor` の「正面の方位」。`project.head_azimuth` は**受音点ごとに持てる**）。上下は扱わない（実務では水平面で足りる）

いま残っている論点

- **1 ウィンドウ完結の GUI**（TODO G-7）。いまは必要なウィンドウを順に開く形
- **複数音源**（TODO F-7b）。音線追跡までは入っているが、パイプライン全体はまだ 1 音源前提
- **面の確認画面の操作性**（2026-08-23 にユーザーから「今の感じでは使い辛い」）。

  提案は出してあり、どれを入れるかは判断待ち
- **1/3 オクターブ展開が入力の範囲より外に中身を作る**（TODO E-15）